

# Groove Intern Assignment -

Build a small service that integrates with the Meta Marketing API, fetches ads-related data, and stores it in a persistent datastore for later querying or inspection.

This assignment is intentionally open-ended. You are expected to figure out:

- what access and configuration are required to use the Meta Marketing API
- which endpoints and entities are relevant
- how the data should be modeled and stored
- how someone should run and verify your solution

Requirements:

- Fetch data from the Meta Marketing API
- Store the fetched data in a database or datastore of your choice
- Include clear setup and run instructions
- Document your assumptions, design choices, and limitations

If you need access token details, account information, or help with credentials, contact me.

For Reference -

<https://developers.facebook.com/documentation/ads-commerce/marketing-api/reference>

Follow Up -

**Objective:** Build a basic command-line or web-based chatbot that allows a user to query the ads data collected in Part 1 using natural language.

## The Challenge

Now that you have persistent data in your datastore, your goal is to integrate a Large Language Model (LLM) to act as an interface. Instead of writing SQL queries, a user should be able to ask: *"Which campaign had the highest ROI last week?"* or *"How much did we spend on Tuesday?"*

## Requirements:

1. **LLM Integration:** Use an LLM of your choice (OpenAI, Anthropic, or a local model via Ollama) to interpret user queries.
2. **Context Injection:** Implement a way for the LLM to understand your database schema (e.g., using a **SQL Agent** or a **Text-to-SQL** prompt strategy).
3. **Natural Language Response:** The bot should execute the necessary query and return a human-readable summary of the data.
4. **Error Handling:** How does the bot handle a query for data that doesn't exist?

## Technical Constraints:

- You may use frameworks like **LangChain**, **LlamaIndex**, or build a custom solution using raw LLM APIs.
- The solution should be "plug-and-play" with the database created in Part 1.

## Documentation to Include:

- **Prompt Engineering:** Explain the system prompt you used to ensure the LLM generates valid queries.
- **Security:** Briefly discuss the risks of "SQL Injection" when letting an LLM generate queries and how you might mitigate them.
- **Scaling:** How would you handle a database with millions of rows where you cannot pass all the data in the prompt context?